

3D Gaussian Splatting 与传统光栅化管线的混合渲染技术： 系统集成与性能探究

金泽辉 22521209

2026 年 3 月 28 日

摘 要

近年来，基于 3D Gaussian Splatting (3DGS) 的神经渲染技术在真实场景的高保真视角合成中展现了突破性的进展。然而，原生 3DGS 采用纯前向的、基于计算着色器的 α 混合架构，缺乏传统多边形网格的表面定义。这导致其难以与现有的工业级渲染管线（如延迟渲染、全局光照系统）直接融合。本报告对当前 3DGS 与传统多边形光栅化的混合渲染技术进行了系统调研，并从底层系统开销、深度剔除算法及光照解耦三个维度提出了深入的分析与见解，为未来虚实融合的混合渲染架构提供了优化思路。

关键词： 3D Gaussian Splatting, 混合渲染, 光栅化管线, HZB 遮挡剔除, 光照解耦

1 引言

在现代计算机图形学中，基于多边形（Polygon）的光栅化管线长期占据统治地位。然而，面对极其复杂的自然场景或由摄影测量扫描获得的无结构现实资产，传统的多边形表示往往面临极其庞大的顶点开销和繁琐的拓扑处理。3DGS 提出了一种全新的三维场景表达范式，利用带有协方差矩阵 Σ 和均值 μ 的 3D 高斯椭球来替代传统的离散多边形。其空间密度分布函数可表示为：

$$G(x) = \exp\left(-\frac{1}{2}(x - \mu)^T \Sigma^{-1}(x - \mu)\right)$$

尽管 3DGS 在独立渲染时具备极高的帧率，但在实际工程应用中，纯高斯场景的应用面极为狭窄。我们需要将其引入现有的渲染生态，实现与动态角色、传统物理碰撞体以及复杂后期处理的同屏混合渲染。这一“混合渲染（Hybrid Rendering）”过程不仅面临着渲染管线时序的冲突，更带来了显存带宽、深度一致性及光照计算等一系列底层挑战。

2 混合渲染的理论基础与技术路线综述

要实现混合渲染，首先需要调和两种截然不同的片元生成机制。传统管线依赖 Z-Buffer 实现 $O(1)$ 复杂度的深度测试（Depth Testing），而 3DGS 严格依赖于从后向前的排序（Sorting）与 Alpha 混合。在相机投影阶段，3D 高斯的协方差矩阵 Σ 需要通过仿射变换近似投影到 2D 屏幕空间：

$$\Sigma' = JW\Sigma W^T J^T$$

其中 W 为观察矩阵， J 为射影变换的雅可比矩阵。随后，其核心 α 混合渲染方程可表示为：

$$C = \sum_{i \in \mathcal{N}} c_i \alpha_i \prod_{j=1}^{i-1} (1 - \alpha_j)$$

目前学术界与工业界针对混合渲染主要演化出以下三种技术路线：

2.1 深度感知的管线融合 (Depth-Aware Integration)

这是目前最直接且应用最广的混合方案。其核心思想是在 3DGS 的光栅化阶段（Tile-based Rasterizer）引入传统管线生成的深度缓冲（Depth Buffer）。在计算每个像素的高斯颜色累加前，采样传统网格场景的深度值 Z_{mesh} 。若当前高斯球的深度 $Z_{gaussian} > Z_{mesh}$ ，则认为该高斯被传统几何体遮挡，直接在计算着色器（Compute Shader）中阻断其 α 累加。

- **优势：**实现简单，能够保持 3DGS 原生的渲染高帧率，适用于将 3DGS 作为静态背景的场景。

- **局限：**仅实现了单向剔除（Mesh 遮挡 Gaussian），无法完美处理半透明物体的交叉遮挡，且容易在物体边缘产生锯齿与深度冲突（Z-Fighting）。

2.2 几何提取与延迟管线注入 (Geometry Extraction and Deferred Injection)

为了彻底融入现代引擎的延迟渲染 (Deferred Rendering) 架构, 部分研究 (如 SuGaR 等技术) 致力于从 3DGS 中提取显式的几何信息。通过对高斯分布进行正则化约束, 使其紧贴物体表面, 随后通过密度场的梯度 $\nabla G(x)$ 提取精确的法线 n :

$$n = -\frac{\nabla G(x)}{\|\nabla G(x)\|}$$

从而能够在渲染时不仅输出颜色, 还能输出精确的法线和深度, 构建出伪 G-Buffer。

- **优势：**使得 3DGS 资产能够无缝接入现有的屏幕空间反射 (SSR)、屏幕空间环境光遮蔽 (SSAO) 乃至基于硬件的光线追踪管线。
- **局限：**强制提取表面会破坏 3DGS 表达体积云雾、毛发等半透明材质的优势, 且 G-Buffer 的高频写入极大增加了显存带宽的压力。

2.3 基于粒子系统的前向合成 (Particle-System based Forward Compositing)

在现代游戏引擎中, 3DGS 常被视作一种特殊的海量粒子系统。通过底层图形 API (如 Vulkan 或 DirectX 12) 的 Compute Shader 对高斯数据进行排序和 Splatting, 并在传统管线的半透明渲染阶段 (Translucency Pass) 将其作为一个全屏 Quad 绘制并与主 Color Buffer 混合。

3 分析与见解 (Analysis & Insights)

基于对当前混合渲染管线的调研, 结合计算机系统底层架构与图形学原理, 本报告提出以下三点深度分析:

3.1 遮挡剔除机制的降维与 HZB 优化

目前的混合渲染在处理大量高斯球与传统几何体穿插时, 性能瓶颈往往不在于光栅化本身, 而在于排序 (Sorting) 和无用的内存读取。传统管线为了解决 Overdraw, 广泛采用了分层 Z 缓冲 (Hierarchical Z-Buffer, HZB) 进行遮挡剔除。

在混合渲染架构中, 我们完全可以改变现有的 3DGS 渲染逻辑: 在每一帧对高斯球进行基数排序 (Radix Sort) 之前, 先利用传统管线生成的 HZB 构建一个 Compute Shader 剔除 Pass。算法伪代码如下所示:

Algorithm 1 基于 HZB 的 3D Gaussian 早期遮挡剔除**Require:** HZB 纹理 $\mathcal{H}$, 当前帧高斯集合 $\mathcal{G}$

```

1: for each Gaussian  $g \in \mathcal{G}$  in parallel do
2:   计算屏幕包围盒  $B_{screen}$ , 最大边长  $L$ 
3:   计算 Mip 层级:  $level = \lfloor \log_2(L) \rfloor$ 
4:    $Z_{max\_mesh} \leftarrow \text{Sample}(\mathcal{H}, level, B_{screen})$ 
5:   if  $g.z\_min > Z_{max\_mesh}$  then
6:     Cull: 阻断写入
7:   else
8:     Keep: 压入可见缓冲区
9:   end if
10: end for

```

通过将高斯球投影并与 HZB 深度比对, 可以在早期阶段 (Early-Z) 直接剔除大量被遮挡的高斯实例, 极大降低了排序的时间复杂度与 GPU 显存的带宽开销。

3.2 图形与计算管线的同步开销及其对驱动层的挑战

3DGS 混合渲染在系统工程上的最大痛点是图形管线 (Graphics Pipeline) 与计算管线 (Compute Pipeline) 的频繁上下文切换 (Context Switch)。在现代 GPU 架构下, 传统网格的 G-Buffer 生成依赖固件光栅化器。在混合渲染中, 必须插入严格的内存屏障 (如 Vulkan 中的 `vkCmdPipelineBarrier`) 来确保传统 Depth Buffer 写入完成后, 3DGS 的 Compute Shader 才能开始采样并进行 Splatting。这种同步操作会导致 GPU 渲染流水线出现气泡 (Stalls)。未来的优化需要向驱动级调度靠拢, 例如利用异步计算 (Async Compute), 在网格执行阴影贴图 (Shadow Map) 渲染时, 并行执行 3DGS 的空间哈希与基数排序。

3.3 光照解耦 (Relighting) 的本质矛盾与材质拆解

如果仅仅解决了深度合并, 我们得到的只是一个“拼贴画”。3DGS 在训练时将辐射亮度 (Radiance) 进行了烘焙, 环境的光照已经被编码在了球谐函数 (Spherical Harmonics, SH) 中。当 3DGS 置入带有动态全局光照的引擎 (如 Lumen 系统) 中时, 会出现违和的“双重阴影”现象。真正的混合渲染必须在渲染前对 3DGS 的属性进行解耦, 将 SH 系数拆解为基础反照率、粗糙度和金属度。在混合管线的片元着色阶段, 必须引入标准的 PBR 渲染方程:

$$L_o(\omega_o) = \int_{\Omega} f_r(\omega_i, \omega_o) L_i(\omega_i) (\omega_i \cdot n) d\omega_i$$

实时评估高斯表面的双向反射分布函数 (BRDF) f_r , 这将是混合渲染技术走向工业标准化的最后一块拼图。

4 结论与展望

3D Gaussian Splatting 为计算机图形学带来了前所未有的表达能力，但其完全脱离传统多边形管线的特性也为其工程化应用设置了障碍。本报告详细探讨了当前深度感知混合与 G-Buffer 注入等主流混合渲染方案，并深刻剖析了其在底层硬件调度、HZB 遮挡剔除算法以及物理光照解耦上的核心瓶颈。

可以预见，未来的混合渲染不仅需要算法层面的突破，更需要底层 API 甚至 GPU 硬件架构的协同演进。统一的空间数据结构（如将层次包围盒 BVH 扩展以同时包容网格与高斯体）以及硬件级的 Splatting 单元，将是实现终极虚实融合渲染的必由之路。